

Методы словарей

Словари, как и списки, имеют много полезных методов для упрощения работы с ними и решения повседневных задач. В прошлом уроке мы уже познакомились с тремя словарными методами:

- метод `items()` – возвращает словарные пары `ключ: значение`, как соответствующие им кортежи;
- метод `keys()` – возвращает список ключей словаря;
- метод `values()` – возвращает список значений словаря.

Добавление и изменение элементов в словаре

Чтобы изменить значение по определенному ключу в словаре, достаточно использовать индексацию вместе с оператором присваивания. При этом если ключ уже присутствует в словаре, его значение заменяется новым, если же ключ отсутствует – то в словарь будет добавлен новый элемент.

Приведенный ниже код:

```
info = {'name': 'Sam',  
        'age': 28,  
        'job': 'Teacher'}  
  
info['name'] = 'Timur' # изменяем значение по ключу  
name  
  
info['email'] = 'timyr-guev@yandex.ru' # добавляем в словарь элемент  
с ключом email  
  
print(info)
```

выводит (порядок элементов может отличаться):

```
{'name': 'Timur', 'age': 28, 'job': 'Teacher', 'email': 'timyr-  
guev@yandex.ru'}
```

Обратите внимание на отличие в поведении словарей и списков:

- Если в списке `lst` нет элемента с индексом `7`, то попытка обращения к нему, например, с помощью строки кода `print(lst[7])` приведет к возникновению ошибки. И попытка присвоить ему значение `lst[7] = 100` тоже приведет к возникновению ошибки.
- Если в словаре `dct` нет элемента с ключом `name`, то попытка обращения к нему, например, с помощью строки кода `print(dct['name'])` приведет к возникновению ошибки. Однако попытка присвоить значение по отсутствующему ключу `dct['name'] = 'Timur'` ошибки не вызовет.

Решим следующую задачу: пусть задан список чисел `numbers`, где некоторые числа встречаются неоднократно. Нужно узнать, сколько именно раз встречается каждое из чисел.

```
numbers = [9, 8, 32, 1, 10, 1, 10, 23, 1, 4, 10, 4, 2, 2, 2, 2, 1, 10, 1, 2, 2, 32, 23, 23]
```

Первый код, который приходит в голову, имеет вид:

```
numbers = [9, 8, 32, 1, 10, 1, 10, 23, 1, 4, 10, 4, 2, 2, 2, 2, 1, 10, 1, 2, 2, 32, 23, 23]
```

```
result = {}
```

```
for num in numbers:
```

```
    result[num] += 1
```

Однако просто так сделать `result[num] += 1` нельзя, так как если ключа `num` в словаре еще нет, то возникнет ошибка `KeyError`.

Следующий программный код полностью решает поставленную задачу:

```
numbers = [9, 8, 32, 1, 10, 1, 10, 23, 1, 4, 10, 4, 2, 2, 2, 2, 1, 10, 1, 2, 2, 32, 23, 23]
```

```
result = {}
```

```
for num in numbers:
```

```
    if num not in result:
```

```
        result[num] = 1
```

```
    else:
```

```
        result[num] += 1
```

Цикл `for` перебирает все элементы списка `numbers` и для каждого проверяет, присутствует ли он уже в качестве ключа в словаре `result`. Если значение отсутствует, значит, число встретилось впервые и мы иницилируем значение `result[num] = 1`. Если значение уже присутствует в словаре, увеличим `result[num]` на единицу.

Этот код можно улучшить с помощью метода `get()`.

Метод `get()`

Мы можем получить значение в словаре по ключу с помощью индексации, то есть квадратных скобок. Однако, как мы знаем, в случае отсутствия ключа будет происходить ошибка `KeyError`.

Приведенный ниже код:

```
info = {'name': 'Bob',  
        'age': 25,  
        'job': 'Dev'}  
print(info['name'])
```

ВЫВОДИТ:

Bob

Приведенный ниже код:

```
info = {'name': 'Bob',  
        'age': 25,  
        'job': 'Dev'}  
print(info['salary'])
```

приводит к возникновению ошибки:

`KeyError: 'salary'`

Для того чтобы избежать возникновения ошибки в случае отсутствия ключа в словаре, можно использовать метод `get()`, способный кроме ключа принимать и второй аргумент — значение, которое вернется, если заданного ключа нет. Когда второй аргумент не указан, то метод в случае отсутствия ключа возвращает `None`.

Приведенный ниже код:

```
info = {'name': 'Bob',  
        'age': 25,  
        'job': 'Dev'}  
item1 = info.get('salary')  
item2 = info.get('salary', 'Информации о зарплате нет')  
print(item1)  
print(item2)
```

ВЫВОДИТ:

None

Информации о зарплате нет

С помощью словарного метода `get()` можно упростить код в задаче о повторяющихся числах.

```
numbers = [9, 8, 32, 1, 10, 1, 10, 23, 1, 4, 10, 4, 2, 2, 2, 2, 1, 10, 1, 2, 2, 32, 23, 23]
```

```
result = {}
```

```
for num in numbers:
```

```
    result[num] = result.get(num, 0) + 1
```

Цикл `for` перебирает все элементы списка `numbers` и для каждого элемента с помощью метода `get()` мы получаем либо его значение из словаря `result`, либо значение по умолчанию, равное `0`. К данному значению прибавляется единица, и результат записывается обратно в словарь по нужному ключу.

Метод `update()`

Метод `update()` реализует своеобразную операцию конкатенации для словарей. Он объединяет ключи и значения одного словаря с ключами и значениями другого. При совпадении ключей в итоге сохранится значение словаря, указанного в качестве аргумента метода `update()`.

Приведенный ниже код:

```
info1 = {'name': 'Bob',
        'age': 25,
        'job': 'Dev'}
info2 = {'age': 30,
        'city': 'New York',
        'email': 'bob@web.com'}

info1.update(info2)

print(info1)
```

Выводит (порядок элементов может отличаться):

```
{'name': 'Bob', 'age': 30, 'job': 'Dev', 'city': 'New York', 'email': 'bob@web.com'}
```

В Python 3.9 появились операторы `|` и `|=`, которые реализуют операцию конкатенации словарей.

Приведенный ниже код:

```
info1 = {'name': 'Bob',  
        'age': 25,  
        'job': 'Dev'}  
  
info2 = {'age': 30,  
        'city': 'New York',  
        'email': 'bob@web.com'}  
  
info1 |= info2  
  
print(info1)
```

аналогичен предыдущему коду.

Метод `setdefault()`

Метод `setdefault()` позволяет получить значение из словаря по заданному ключу, автоматически добавляя элемент словаря, если он отсутствует.

Метод принимает два аргумента:

- `key`: ключ, значение по которому следует получить, если таковое имеется в словаре, либо создать.
- `default`: значение, которое будет использовано при добавлении нового элемента в словарь.

В зависимости от значений параметров `key` и `default` возможны следующие сценарии работы данного метода.

Сценарий 1. Если ключ `key` присутствует в словаре, то метод возвращает значение по заданному ключу (независимо от того, передан параметр `default` или нет).

Приведенный ниже код:

```
info = {'name': 'Bob',  
        'age': 25}  
  
name1 = info.setdefault('name')           # параметр default не задан  
  
name2 = info.setdefault('name', 'Max')    # параметр default задан
```

```
print(name1)
```

```
print(name2)
```

ВЫВОДИТ:

```
Bob
```

```
Bob
```

Сценарий 2. Если ключ `key` отсутствует в словаре, то метод вставляет переданное значение `default` по заданному ключу.

Приведенный ниже код:

```
info = {'name': 'Bob',  
        'age': 25}  
  
job = info.setdefault('job', 'Dev')  
  
print(info)  
print(job)
```

ВЫВОДИТ:

```
{'name': 'Bob', 'age': 25, 'job': 'Dev'}  
  
Dev
```

При этом если значение `default` не передано в метод, то вставится значение `None`.

Приведенный ниже код:

```
info = {'name': 'Bob',  
        'age': 25}  
  
job = info.setdefault('job')  
  
print(info)  
print(job)
```

ВЫВОДИТ:

```
{'name': 'Bob', 'age': 25, 'job': None}  
  
None
```

Удаление элементов из словаря

Существует несколько способов удаления элементов из словаря:

- оператор `del`;
- метод `pop()`;
- метод `popitem()`;
- метод `clear()`.

Оператор del

С помощью оператора `del` можно удалять элементы словаря по определенному ключу.

Следующий программный код:

```
info = {'name': 'Sam',  
        'age': 28,  
        'job': 'Teacher',  
        'email': 'timyr-guev@yandex.ru'}  
  
del info['email'] # удаляем элемент имеющий ключ email  
  
del info['job']   # удаляем элемент имеющий ключ job  
  
print(info)
```

выводит (порядок элементов может отличаться):

```
{'name': 'Sam', 'age': 28}
```

Если удаляемого ключа в словаре нет, возникнет ошибка `KeyError`.

Метод pop()

Оператор `del` удаляет из словаря элемент по заданному ключу вместе с его значением. Если требуется получить само значение удаляемого элемента, то нужен метод `pop()`.

Следующий программный код:

```
info = {'name': 'Sam',  
        'age': 28,  
        'job': 'Teacher',  
        'email': 'timyr-guev@yandex.ru'}
```

```
email = info.pop('email')          # удаляем элемент по ключу email,  
возвращая его значение
```

```
job = info.pop('job')              # удаляем элемент по ключу job,  
возвращая его значение
```

```
print(email)
```

```
print(job)
```

```
print(info)
```

ВЫВОДИТ:

```
timyr-guev@yandex.ru
```

```
Teacher
```

```
{'name': 'Sam', 'age': 28}
```

Единственное отличие этого способа удаления от оператора `del` — он возвращает удаленное значение. В остальном этот способ идентичен оператору `del`. В частности, если удаляемого ключа в словаре нет, возникнет ошибка `KeyError`.

Чтобы ошибка не появлялась, этому методу можно передать второй аргумент. Он будет возвращен, если указанного ключа в словаре нет. Это позволяет реализовать безопасное удаление элемента из словаря:

```
surname = info.pop('surname', None)
```

Если ключа `surname` в словаре нет, то в переменной `surname` будет храниться значение `None`.

Метод `popitem()`

Метод `popitem()` удаляет из словаря **последний добавленный элемент** и возвращает удаляемый элемент в виде кортежа `(ключ, значение)`.

Следующий программный код:

```
info = {'name': 'Bob',
```

```
       'age': 25,
```

```
       'job': 'Dev'}
```

```
info['surname'] = 'Sinclar'
```



```
item = info.popitem()
```

```
print(item)
```

```
print(info)
```

ВЫВОДИТ:

```
('surname', 'Sinclar')
```

```
{'name': 'Bob', 'age': 25, 'job': 'Dev'}
```

В версиях Python ниже 3.6 метод `popitem()` удалял случайный элемент.

Метод `clear()`

Метод `clear()` удаляет все элементы из словаря.

Следующий программный код:

```
info = {'name': 'Bob',
```

```
       'age': 25,
```

```
       'job': 'Dev'}
```

```
info.clear()
```

```
print(info)
```

Выведет:

```
{}
```

Метод `copy()`

Метод `copy()` создает **поверхностную** копию словаря.

Следующий программный код:

```
info = {'name': 'Bob',
```

```
       'age': 25,
```

```
       'job': 'Dev'}
```

```
info_copy = info.copy()
```

```
print(info_copy)
```

Выведет:

```
{'name': 'Bob', 'age': 25, 'job': 'Dev'}
```

Не стоит путать копирование словаря (метод `copy()`) и присвоение новой переменной ссылки на старый словарь.

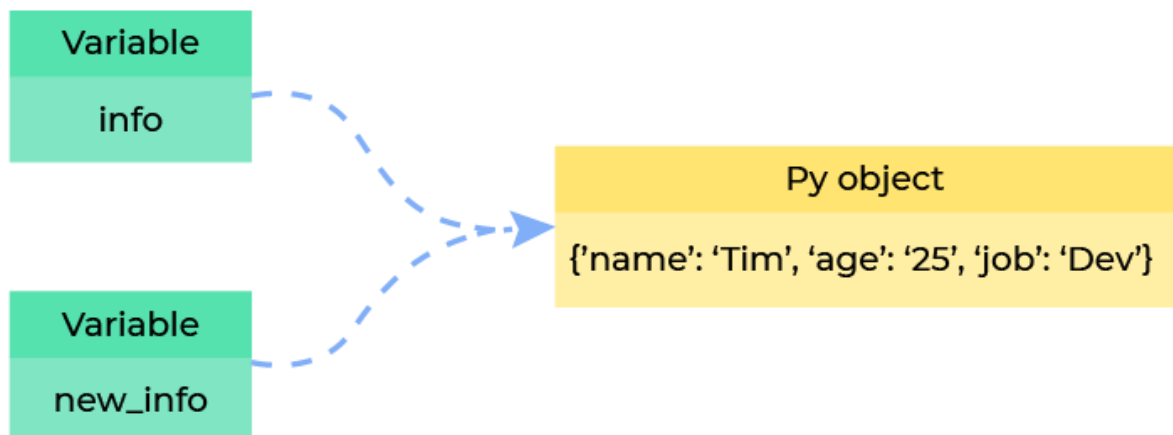
Следующий программный код:

```
info = {'name': 'Bob',  
        'age': 25,  
        'job': 'Dev'}  
  
new_info = info  
  
new_info['name'] = 'Tim'  
  
print(info)
```

Выводит:

```
{'name': 'Tim', 'age': 25, 'job': 'Dev'}
```

Оператор присваивания (`=`) не копирует словарь, а лишь присваивает ссылку на старый словарь новой переменной.



Таким образом, когда мы изменяем словарь `new_info`, меняется и словарь `info`. Если необходимо изменить один словарь, не изменяя второй, используют метод `copy()`.

Следующий программный код:

```
info = {'name': 'Bob',
```

```
    'age': 25,  
    'job': 'Dev'}  
  
new_info = info.copy()  
new_info['name'] = 'Tim'  
  
print(info)  
print(new_info)
```

ВЫВОДИТ:

```
{'name': 'Bob', 'age': 25, 'job': 'Dev'}  
{'name': 'Tim', 'age': 25, 'job': 'Dev'}
```

Примечания

Примечание 1. Словарь можно использовать вместо нескольких вложенных условий, если вам нужно проверить число на равенство. Например, вместо

```
num = int(input())  
  
if num == 1:  
    description = 'One'  
  
elif num == 2:  
    description = 'Two'  
  
elif num == 3:  
    description = 'Three'  
  
else:  
    description = 'Unknown'  
  
print(description)
```

МОЖНО НАПИСАТЬ

```
num = int(input())  
  
description = {1: 'One', 2: 'Two', 3: 'Three'}  
  
print(description.get(num, 'Unknown'))
```

На практике такой код встречается достаточно часто, особенно если в программе необходимо часто осуществлять проверку указанного типа.

Примечание 2. Методы `.get()` и `.setdefault()` в Python используются для работы со словарями, но они выполняют разные функции.

`.get(key, default):`

- возвращает значение `default`, если ключ отсутствует в словаре (по умолчанию `None`, если `default` не указан);
- не изменяет сам словарь.

Пример:

```
my_dict = {'a': 1, 'b': 2}
```

```
value = my_dict.get('a', 0) # Получаем значение по ключу 'a', если  
ключ отсутствует, возвращаем 0
```

`.setdefault(key, default):`

- если ключ отсутствует в словаре, вставляет новую пару ключ-значение с ключом `key` и значением `default` в словарь, затем возвращает `default`.

Пример:

```
my_dict = {'a': 1, 'b': 2}
```

```
value = my_dict.setdefault('c', 0) # Получаем значение по ключу 'c',  
если ключ отсутствует, вставляем 'c': 0
```

Важное отличие: `.get()` только получает значение, не меняя словарь, в то время как `.setdefault()` может изменить словарь, если ключ отсутствует.